Advanced Machine Learning, ETH Zurich

Language models

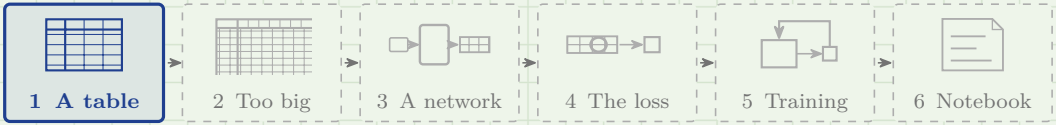
From a table to a neural network

Carlos Cotrini

Autumn semester 2026

Entry 1

A language model is a table



The question. A chatbot writes its answer one word at a time. How does it choose each word?

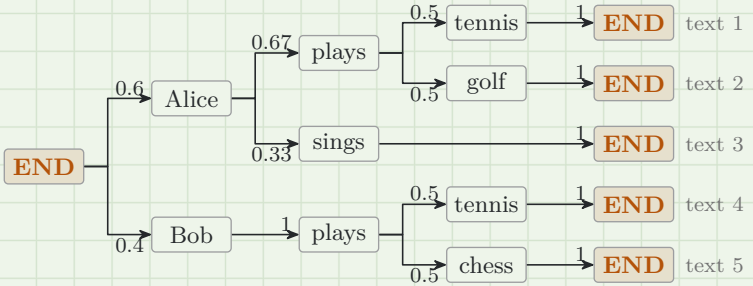
Now. A language model is a table of chances. We draw five short texts as a tree of chances, turn the tree into a table, fill it in by counting, and use it to write texts and to answer a question.

A language model learns from a collection of texts, its *corpus*. Ours holds five short texts, each about what Alice or Bob likes to do.

text 1	Alice plays tennis
text 2	Alice plays golf
text 3	Alice sings
text 4	Bob plays tennis
text 5	Bob plays chess

A real corpus holds billions of texts: books, articles, web pages. Ours is small enough to follow every step on a slide.

Draw the five texts as a tree: they start together and fork where their words differ. A branch gets the share of the texts that take it: three of the five start with Alice, so 0.6.



The root is **END**: a text is about to start.

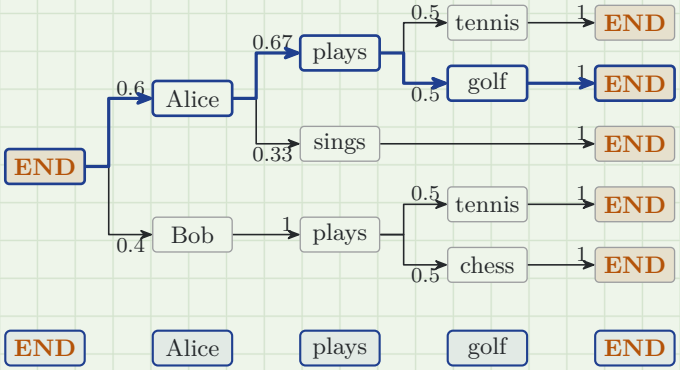
Alice starts 3 of the 5 texts, Bob 2.

After Alice: plays 2 of 3, sings 1 of 3. After Bob: plays 2 of 2.

After plays: 1 of 2 each way.

Every text ends with **END**: each path is one text.

Start at the root, **END**. At each fork pick a branch at random, as often as its chance says: Alice six times in ten, Bob four. Write down its word, and stop at an **END**.



Start at the root, **END**.

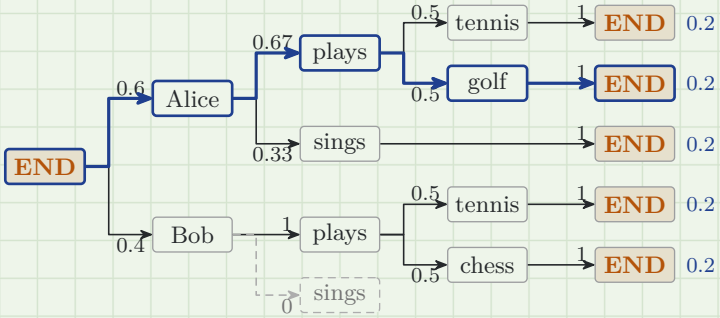
Alice or Bob? This time Alice, with chance 0.6.

After Alice: plays or sings? This time plays, 0.67.

After plays: golf or tennis? This time golf, 0.5.

After golf, **END** for sure: stop. The text is Alice plays golf.

Of all walks, 3 in 5 take Alice, and 2 in 3 of those go on to plays. So the chance of a whole text is the product of the chances along its path.



One walk in five writes Alice plays golf.

Each of the five texts gets 0.2, and the five add up to 1.

Bob sings gets 0: no text goes from Bob to sings. A walk writes the five texts and no other.

Alice plays golf: $\frac{3}{5} \times \frac{2}{3} \times \frac{1}{2} \times 1 = \frac{1}{5}$

To turn the tree into a table, first write the corpus into one column, one word per cell, text after text, with **END** before the first text, between every two texts and after the last.

corpus
END
Alice
plays
tennis
END
Alice
plays
golf
END
Alice
sings
END
Bob
plays
tennis
END
Bob
plays
chess
END

Alice plays tennis

Alice plays golf

Alice sings

Bob plays tennis

Bob plays chess

We read text 1 word by word, from the top down, with an **END** before it and after it.

Then text 2, and the **END** after it.

Twenty words in all. **END** marks where one text stops and the next one starts: it counts as a word of its own.

Each row will ask one question: which word comes next? So every word of the corpus gets a row, numbered from the top, except the last **END**, after which no word comes.

	corpus
1	END
2	Alice
3	plays
4	tennis
5	END
6	Alice
7	plays
8	golf
9	END
10	Alice
11	sings
12	END
13	Bob
14	plays
15	tennis
16	END
17	Bob
18	plays
19	chess
	END

Each word of the column becomes a row.

The rows are numbered from the top:
rows 1 to 19.

The last **END** ends the corpus: no row.

A row stands for the text written so far, its *prefix*: start at the row and read upward until you meet an **END**, both included.

	corpus
1	**END**
2	Alice
3	plays
4	tennis
5	**END**
6	Alice
7	plays
8	golf
9	**END**
10	Alice
11	sings
12	**END**
13	Bob
14	plays
15	tennis
16	**END**
17	Bob
18	plays
19	chess
	END

Row 8: its prefix is **END** Alice plays golf.

Row 14: its prefix is **END** Bob plays.

Row 16 is an **END** and its own prefix, **END**: a text is about to start.

The *dictionary* lists every word of the corpus once, **END** included: eight words. The table puts one column per word of the dictionary beside the corpus column.

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END								
2	Alice								
3	plays								
4	tennis								
5	END								
6	Alice								
7	plays								
8	golf								
9	END								
10	Alice								
11	sings								
12	END								
13	Bob								
14	plays								
15	tennis								
16	END								
17	Bob								
18	plays								
19	chess								
	END								

the dictionary

One row for each word of the corpus: 19 rows.

One column for each word of the dictionary: 8 columns.

Each cell answers one question: how likely is the word of its column to come right after the prefix of its row?

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END								
2	Alice								
3	plays								
4	tennis								
5	END								
6	Alice								
7	plays								
8	golf								
9	END								
10	Alice								
11	sings								
12	END								
13	Bob								
14	plays								
15	tennis								
16	END								
17	Bob								
18	plays								
19	chess								
	END								

How likely is golf right after **END** Alice plays?

How likely is Bob right after **END**?

How likely is plays right after **END** Bob plays tennis?

Subject

Count what came after the rows whose prefix is END

Entry 1

11

The word that came after a row is the word in the row below it. Take the rows whose prefix is **END**, count the words below them, and divide by the number of rows.

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END	0.6	0.4						
2	Alice								
3	plays								
4	tennis								
5	END	0.6	0.4						
6	Alice								
7	plays								
8	golf								
9	END	0.6	0.4						
10	Alice								
11	sings								
12	END	0.6	0.4						
13	Bob								
14	plays								
15	tennis								
16	END	0.6	0.4						
17	Bob								
18	plays								
19	chess								
	END								

Rows 1, 5, 9, 12 and 16 have the prefix **END**.

The word below each of them: Alice, Alice, Alice, Bob, Bob.

Alice 3 of 5 = 0.6

Bob 2 of 5 = 0.4

All five rows have the same prefix, so all five get Alice 0.6 and Bob 0.4.

Now the prefix **END** Alice: count the words below its rows, as before. The same count for every other prefix fills the table.

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END	0.6	0.4						
2	Alice			0.67	0.33				
3	plays						0.5	0.5	
4	tennis								1
5	END	0.6	0.4						
6	Alice			0.67	0.33				
7	plays						0.5	0.5	
8	golf								1
9	END	0.6	0.4						
10	Alice			0.67	0.33				
11	sings								1
12	END	0.6	0.4						
13	Bob			1					
14	plays					0.5		0.5	
15	tennis								1
16	END	0.6	0.4						
17	Bob			1					
18	plays					0.5		0.5	
19	chess								1
	END								

Rows 2, 6 and 10 have the prefix **END** Alice.

The word below each: plays, plays, sings.

plays 2 of 3 = 0.67

sings 1 of 3 = 0.33

The same count for each of the eight other prefixes fills the table. A blank cell holds 0.

In the full table, rows with the same prefix are equal. Rows with the same word after different prefixes can differ.

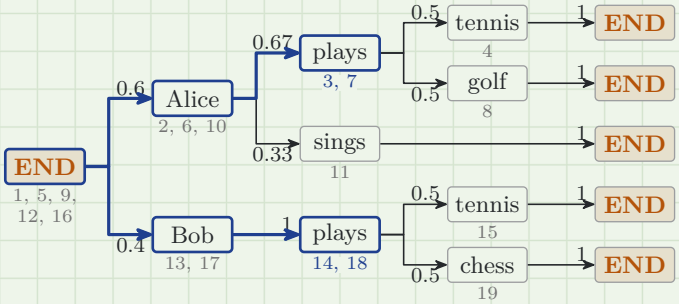
	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END	0.6	0.4						
2	Alice			0.67	0.33				
3	plays						0.5	0.5	
4	tennis								1
5	END	0.6	0.4						
6	Alice			0.67	0.33				
7	plays						0.5	0.5	
8	golf								1
9	END	0.6	0.4						
10	Alice			0.67	0.33				
11	sings								1
12	END	0.6	0.4						
13	Bob			1					
14	plays					0.5		0.5	
15	tennis								1
16	END	0.6	0.4						
17	Bob			1					
18	plays					0.5		0.5	
19	chess								1
	END								

Equal prefixes: the five **END** rows all read Alice 0.6, Bob 0.4.

Rows 3 and 14 both hold plays. Their prefixes differ, and so do their rows.

A row needs the whole prefix: the last word alone is too little.

A fork of the tree is a prefix: the words on the path from the root to it. The rows of the table with that prefix all sit at that fork, and hold the chances of its branches.



	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
3	plays						0.5	0.5	
14	plays					0.5		0.5	

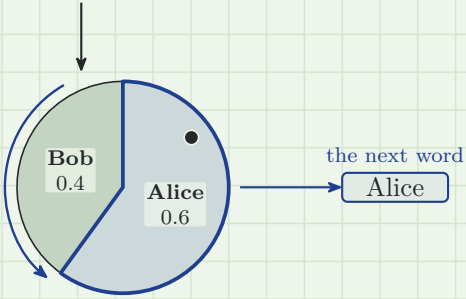
Under each fork, its rows of the table. The **END** rows start a text: they sit at the root.

Rows 3 and 14 both hold plays, at two forks: **END** Alice plays and **END** Bob plays.

Their branches differ, and so do their rows of the table.

To pick the word after a prefix, turn its row into a roulette: each word gets a slice as big as its chance. The slices fill the whole wheel, because a row adds up to 1.

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END	0.6	0.4						

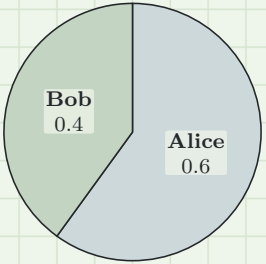


Alice gets six tenths of the wheel and Bob four tenths. A blank cell is a chance of 0 and gets no slice.

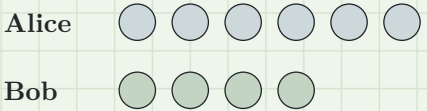
Spin the wheel.

The ball lands in the slice of Alice: the next word is Alice.

Spin the roulette of the prefix **END** ten times. Alice comes out about six times and Bob about four, since the slice of Alice is six tenths of the wheel.



Ten spins, on average



A real run of ten spins can differ a little. Over many spins, the shares come close to 0.6 and 0.4.

Start from the prefix **END**. Find a row with that prefix, spin its roulette and write down the word that comes out. The text so far is the new prefix: go on until **END** comes out.

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	**END**	0.6	0.4						
2	Alice			0.67	0.33				
3	plays						0.5	0.5	
4	tennis								1
5	**END**	0.6	0.4						
6	Alice			0.67	0.33				
7	plays						0.5	0.5	
8	golf								1
9	**END**	0.6	0.4						
10	Alice			0.67	0.33				
11	sings								1
12	**END**	0.6	0.4						
13	Bob			1					
14	plays					0.5		0.5	
15	tennis								1
16	**END**	0.6	0.4						
17	Bob			1					
18	plays					0.5		0.5	
19	chess								1
	END								

END: row 1. Spin: Alice.

END Alice: row 2. Spin: plays.

END Alice plays: row 3. Spin: golf.

END Alice plays golf: row 8. Spin: **END**.

END came out. The text is Alice plays golf, one of the five texts of the corpus.

Put a question in front of each text, as in a chat: What does Alice like? Alice plays tennis. Read the five texts into one column, with **END** between them, and count as before.

What does Alice like?
Alice plays tennis

What does Alice like?
Alice plays golf

What does Alice like?
Alice sings

What does Bob like?
Bob plays tennis

What does Bob like?
Bob plays chess

	corpus
1	END
2	What
3	does
4	Alice
5	like?
6	Alice
7	plays
8	tennis
9	END

Row 5: its prefix is **END** What does Alice like?, the question.

Ask the chatbot: What does Alice like? Your question becomes the prefix **END** What does Alice like?, and the table continues it.

The prefix is **END** and the question. Spin its row and append the word that comes out; then spin the row of the longer prefix, and so on, until **END** comes out.

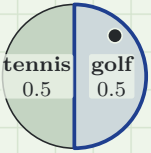
END What does Alice like?



END What does Alice like? Alice



END What does Alice like? Alice plays



END What does Alice like? Alice plays golf

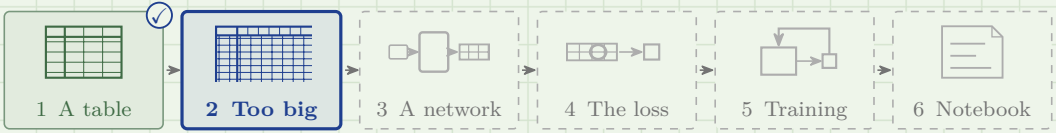


END means stop

The answer: Alice plays golf, one spin for each word

Entry 2

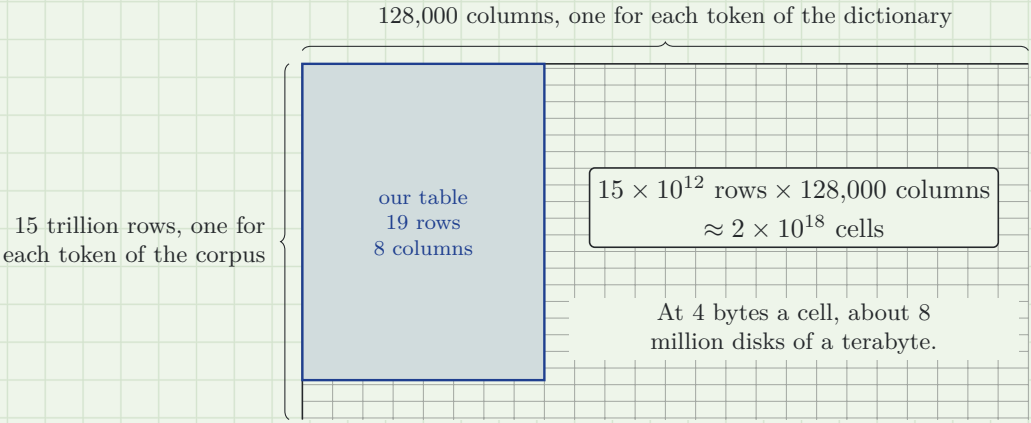
The table is too big



So far. A language model is a table: one row per word of the corpus, one column per word of the dictionary, filled in by counting. Spinning its rows writes texts and answers questions.

Now. A real table is far too big to store. So we compute each row only when we need it.

Meta trained its language model Llama 3 on over 15 trillion *tokens*, words and pieces of words, with a dictionary of 128,000 tokens. Our layout gives that corpus a table.



A chatbot must also answer prompts that nobody wrote before. So its table needs a row for every prompt one could type, each prompt a prefix after **END**.

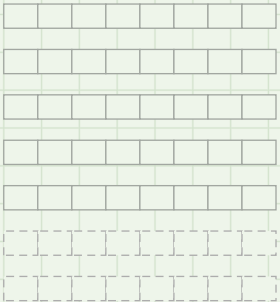
END What does Alice like?

END Write a poem about Zurich

END Summarize this report

END Translate this email

END Plan a team offsite

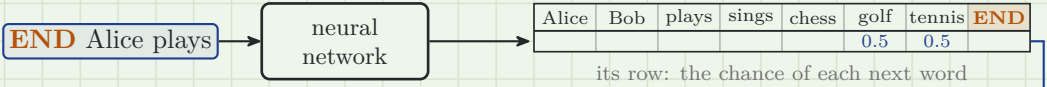


A dictionary of 100,000 words and prompts of 20 words give

$$100{,}000^{20} = 10^{100} \text{ prompts.}$$

The universe holds about 10^{80} atoms. The table would need more rows than there are atoms in the universe.

Store no table at all. A neural network takes the prefix of a row and computes its row, the chance of every next word, at the moment we need it.



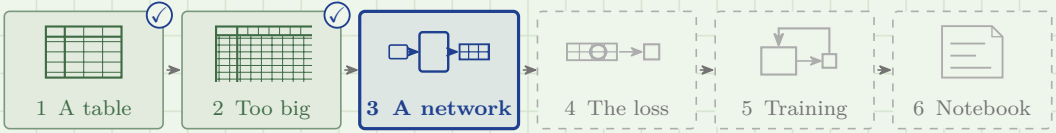
That row is row 7 of our table, whose prefix is **END** Alice plays.

The table is never stored: the network computes any row from its prefix, when it is needed.

	corpus	Alice	Bob	plays	sings	chess	golf	tennis	END
1	END								
2	Alice								
3	plays								
4	tennis								
5	END								
6	Alice								
7	plays						0.5	0.5	
8	golf								

Entry 3

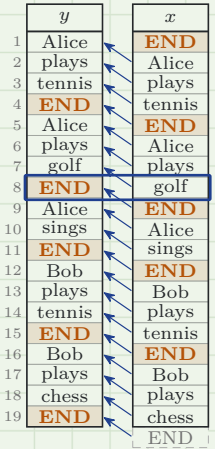
A network computes the table



So far. A language model is a table of chances: one row per word of the corpus, one column per word of the dictionary. A real table is far too big to store.

Now. A neural network computes the table. It reads the corpus, turns every word into numbers, and gives every row its chances.

The corpus column of part 1 gets a name, x: row i holds the word x_i. Beside it goes a second column, y, with the word that came after each row.



Row i of x holds the word x_i. The last **END** gets no row: no word comes after it.

To make y, take the corpus, drop its first **END** and move every word up one row.

So y_i is the word that came right after x_i. Row 8 holds golf in x and **END** in y: that text ended after golf.

The network reads the column x and writes a table, $\hat{Y}$: for every row of x, the chance of each word of the dictionary coming next.

	y
1	Alice
2	plays
3	tennis
4	END
5	Alice
6	plays
7	golf
8	END
9	Alice
10	sings
11	END
12	Bob
13	plays
14	tennis
15	END
16	Bob
17	plays
18	chess
19	END

x
END
Alice
plays
tennis
END
Alice
plays
golf
END
Alice
sings
END
Bob
plays
tennis
END
Bob
plays
chess



Alice	Bob	plays	sings	chess	golf	tennis	END
					0.5	0.5	

The network reads all of x.

$\hat{Y}$: $n = 19$ rows, one per row of x, and 8 columns, one per word.

Row 7 should be the row of our table: golf 0.5 and tennis 0.5.

Each entry of $\hat{Y}$ answers the question of part 1: how likely is this word to come right after this prefix?

	y	x
1	Alice	**END**
2	plays	Alice
3	tennis	plays
4	**END**	tennis
5	Alice	**END**
6	plays	Alice
7	golf	plays
8	**END**	golf
9	Alice	**END**
10	sings	Alice
11	**END**	sings
12	Bob	**END**
13	plays	Bob
14	tennis	plays
15	**END**	tennis
16	Bob	**END**
17	plays	Bob
18	chess	plays
19	**END**	chess

Alice	Bob	plays	sings	chess	golf	tennis	**END**
					0.5	0.5	

Row 7 stands for its prefix, **END** Alice plays: rows 5 to 7 of x.

Column 6 stands for one word of the dictionary, golf.

Their entry $\hat{y}_{ij}$: the chance of word j right after the prefix of row i. Here $\hat{y}_{7,\text{golf}} = 0.5$.

Every row is a roulette, so its chances add up to 1:

$$\sum_j \hat{y}_{ij} = 1$$

A network computes with numbers. So every word of the dictionary gets a row of d numbers, its *embedding*; together the rows form the embedding table.

$d = 3$ numbers

one row for each word of the dictionary

word	embedding		
Alice	0.1	0.1	0.7
Bob	−0.8	0.5	0.2
plays	−0.4	−0.4	−0.3
sings	−0.2	0.9	−0.6
chess	−0.8	−0.4	0.3
golf	0.8	−0.1	−0.2
tennis	−0.3	0.9	−0.2
END	0.5	−0.6	0.2

Each of the 8 words gets its own row of $d = 3$ numbers, shaded in a tint of its own.

At first the numbers are guessed: small, and at random.

Training moves them, as it moves every weight of the network: the embeddings are learned.

We use $d = 3$ to fit the slide. The notebook uses $d = 16$; Llama 3 uses 4,096 in its small model and 8,192 in its large one.

Replace every word of x by its row of the embedding table. The result is E, a block with one row for every row of x and d columns.

	y	x	E		
1	Alice	**END**	0.5	−0.6	0.2
2	plays	Alice	0.1	0.1	0.7
3	tennis	plays	−0.4	−0.4	−0.3
4	**END**	tennis	−0.3	0.9	−0.2
5	Alice	**END**	0.5	−0.6	0.2
6	plays	Alice	0.1	0.1	0.7
7	golf	plays	−0.4	−0.4	−0.3
8	**END**	golf	0.8	−0.1	−0.2
9	Alice	**END**	0.5	−0.6	0.2
10	sings	Alice	0.1	0.1	0.7
11	**END**	sings	−0.2	0.9	−0.6
12	Bob	**END**	0.5	−0.6	0.2
13	plays	Bob	−0.8	0.5	0.2
14	tennis	plays	−0.4	−0.4	−0.3
15	**END**	tennis	−0.3	0.9	−0.2
16	Bob	**END**	0.5	−0.6	0.2
17	plays	Bob	−0.8	0.5	0.2
18	chess	plays	−0.4	−0.4	−0.3
19	**END**	chess	−0.8	−0.4	0.3

word	**embedding**		
Alice	0.1	0.1	0.7
Bob	−0.8	0.5	0.2
plays	−0.4	−0.4	−0.3
sings	−0.2	0.9	−0.6
chess	−0.8	−0.4	0.3
golf	0.8	−0.1	−0.2
tennis	−0.3	0.9	−0.2
END	0.5	−0.6	0.2

the embedding table

Row 1 holds **END**: it takes the row of **END**.

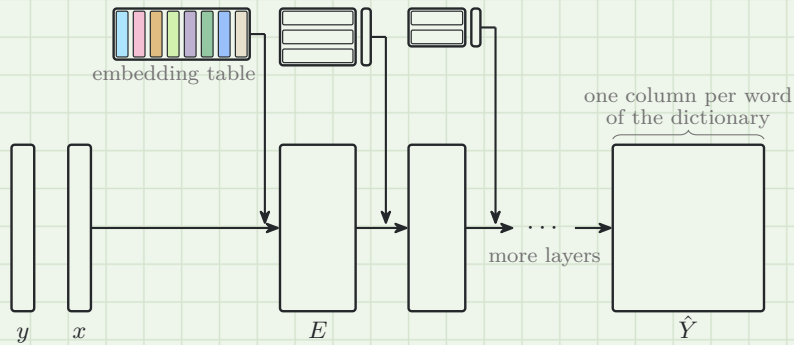
Row 2 holds Alice: it takes the row of Alice, and its tint.

Every row does the same: E has $n = 19$ rows and $d = 3$ columns.

Equal words give equal rows, in one tint: the four rows of plays, the five of **END**.

Every box has the shape of its data: y and x are columns, E is n rows by d columns. Above each block sit the weights that work on it, on their side.

The embedding table is a block of weights too: it turns x into E.



A layer: its block times its weights, plus the biases, then a ReLU. Out comes a new block, a row per row of x.

The last layer gives each row 8 numbers. The softmax makes them chances that add up to 1: $\hat{Y}$.

The last layer gives every row 8 numbers, one for each word of the dictionary: its *scores* s_{ij}. We follow row 7, whose prefix is **END** Alice plays.

	Alice	Bob	plays	sings	chess	golf	tennis	END
score s_{7j}	−1.2	−2.0	−2.4	−1.7	0.3	4.0	4.0	−0.4
any number								

Row 7 gets one score per word: any real number, positive or negative.

A bigger score favours its word. A trained network gives golf and tennis, the words that came after **END** Alice plays, the biggest: 4.0 each.

A row of chances must be positive and add up to 1. These scores are neither: some are negative, and they add up to 0.6. Two steps turn them into chances.

The first step makes every score positive: each score s is replaced by its exponential, e^s.

	Alice	Bob	plays	sings	chess	golf	tennis	END
score s_{7j} any number	−1.2	−2.0	−2.4	−1.7	0.3	4.0	4.0	−0.4
exp	↓	↓	↓	↓	↓	↓	↓	↓
$e^{s_{7j}}$ positive	0.30	0.14	0.09	0.18	1.35	54.60	54.60	0.67

The exponential turns any number into a positive one.

A big score becomes a big number: $e^{4.0} = 54.60$. A negative score becomes a number close to 0: $e^{-2.4} = 0.09$.

The second step makes the row add up to 1: each of the 8 positive numbers is divided by their sum.

	Alice	Bob	plays	sings	chess	golf	tennis	END	sum
score s_{7j} any number	−1.2	−2.0	−2.4	−1.7	0.3	4.0	4.0	−0.4	
exp $e^{s_{7j}}$ positive	0.30	0.14	0.09	0.18	1.35	54.60	54.60	0.67	111.93
÷ 111.93 chance $\hat{y}_{7j}$ adds up to 1	0.00	0.00	0.00	0.00	0.01	0.49	0.49	0.01	1.00

Add the 8 numbers: their sum is 111.93.

Divide each by 111.93. Golf and tennis get 0.49 each, close to the 0.5 of our table; chess and **END** get 0.01, the others almost nothing. The chances add up to 1.

This is the *softmax*. The network applies it to every row of scores, and out comes $\hat{Y}$.

$$\hat{y}_{ij} = \frac{e^{s_{ij}}}{\sum_k e^{s_{ik}}}$$

Training moves every score, by any amount and either way. Whatever the scores, the softmax gives a row of positive chances that adds up to 1.

before training

	Alice	Bob	plays	sings	chess	golf	tennis	END	sum
score s_{7j}	0	0	0	0	0	0	0	0	
softmax	↓	↓	↓	↓	↓	↓	↓	↓	
chance $\hat{y}_{7j}$	0.125	0.125	0.125	0.125	0.125	0.125	0.125	0.125	1

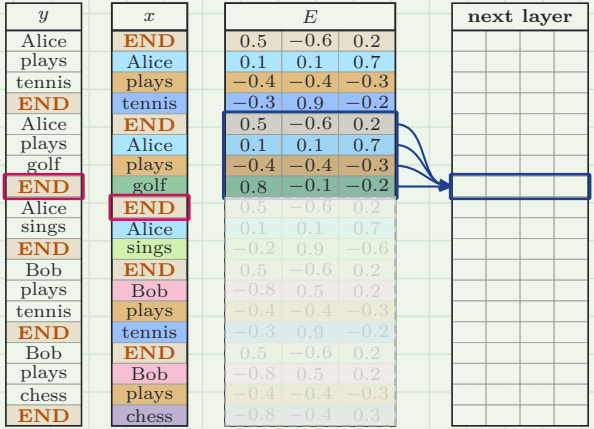
after training

	Alice	Bob	plays	sings	chess	golf	tennis	END	sum
score s_{7j}	−1.2	−2.0	−2.4	−1.7	0.3	4.0	4.0	−0.4	
softmax	↓	↓	↓	↓	↓	↓	↓	↓	
chance $\hat{y}_{7j}$	0.00	0.00	0.00	0.00	0.01	0.49	0.49	0.01	1.00

Before training every score is close to 0. With all 8 at 0, each $e^0 = 1$, the sum is 8, and every chance is $1/8 = 0.125$.

After training, row 7 holds the scores of the last three frames. Either way the row adds up to 1, so training may move any score.

A row of the table depends on its whole prefix, but a row of E holds one word. Which rows of E may row i of the next layer read?



Row 8 of E alone holds golf: too little. Rows 3 and 14 both hold plays, and their rows of the table differ.

So row 8 reads its prefix, rows 5 to 8 of E: **END** Alice plays golf.

Never a row below: row 9 holds **END**, the word that came after golf, y_8. The rows below hold the answers.

Every row reads its own prefix. The notebook does this with one attention layer and a mask.

A network reads a prefix through a window of fixed size. Its *context*, C, is the most rows of its prefix that a row may read, its own row included.

	y	x	E		
1	Alice	**END**	0.5	−0.6	0.2
2	plays	Alice	0.1	0.1	0.7
3	tennis	plays	−0.4	−0.4	−0.3
4	**END**	tennis	−0.3	0.9	−0.2
5	Alice	**END**	0.5	−0.6	0.2
6	plays	Alice	0.1	0.1	0.7
7	golf	plays	−0.4	−0.4	−0.3
8	**END**	golf	0.8	−0.1	−0.2
9	Alice	**END**	0.5	−0.6	0.2
10	sings	Alice	0.1	0.1	0.7
11	**END**	sings	−0.2	0.9	−0.6
12	Bob	**END**	0.5	−0.6	0.2
13	plays	Bob	−0.8	0.5	0.2
14	tennis	plays	−0.4	−0.4	−0.3
15	**END**	tennis	−0.3	0.9	−0.2
16	Bob	**END**	0.5	−0.6	0.2
17	plays	Bob	−0.8	0.5	0.2
18	chess	plays	−0.4	−0.4	−0.3
19	**END**	chess	−0.8	−0.4	0.3

next layer

With $C = 4$, row 8 reads its whole prefix, rows 5 to 8: **END** Alice plays golf.

With $C = 2$, it reads only rows 7 and 8, plays golf. A prefix longer than C is cut to its last C rows.

With $C = 1$, row 8 reads golf alone: only its own row.

Rows 3, 7, 14 and 18 all hold plays. Our table gives them two different rows: golf or tennis after **END** Alice plays, chess or tennis after **END** Bob plays.

	y	x	Alice	Bob	plays	sings	chess	golf	tennis	END
1	Alice	END								
2	plays	Alice								
3	tennis	plays						0.5	0.5	
4	END	tennis								
5	Alice	END								
6	plays	Alice								
7	golf	plays						0.5	0.5	
8	END	golf								
9	Alice	END								
10	sings	Alice								
11	END	sings								
12	Bob	END								
13	plays	Bob								
14	tennis	plays					0.5		0.5	
15	END	tennis								
16	Bob	END								
17	plays	Bob								
18	chess	plays					0.5		0.5	
19	END	chess								

With $C = 1$ all four read plays alone, so they share one row: what came after plays.

With $C = 2$ they read Alice plays or Bob plays: the two rows come back.

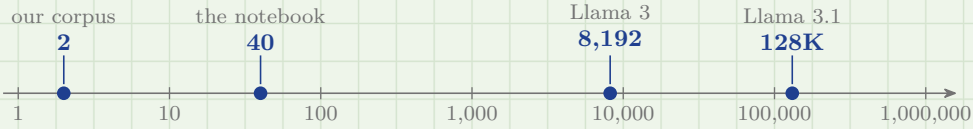
The cost of $C = 1$: after **END** Alice plays, golf drops from 0.5 to 0.25, and chess, which never came after Alice, gets 0.25.

The context is fixed when a network is built. Our corpus needs very little; the notebook and the large language models read much longer prefixes.

Our corpus: its longest prefix has 4 rows, and a context of 2 is enough.

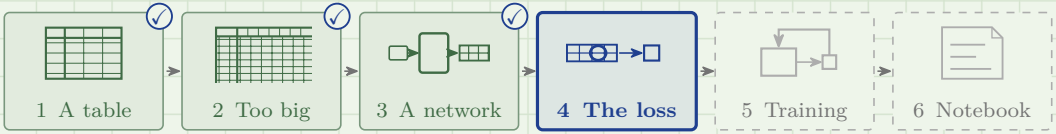
The notebook: a context of 40, the rows of its place table, which part 6 shows. Its longest text has 22 words, so every row reads its whole prefix.

Llama 3 reads up to 8,192 tokens, Llama 3.1 up to 128K. A longer conversation is cut: the chatbot no longer sees its start.



Entry 4

The loss



So far. The network reads x, turns it into E and computes $\hat{Y}$: a row of chances for every row of x, each row reading only its prefix.

Now. How good is $\hat{Y}$? One number, the loss, measures how much chance the network gave the words that really came next.

Suppose $\hat{Y}$ holds the table we counted. In every row, circle the entry in the column of y_i, the word that came next, and carry it into a new column, ℓ.

	y	x	Alice	Bob	plays	sings	chess	golf	tennis	END	ℓ	
1	Alice	END	0.6	0.4							0.51	$= -\log 0.6$
2	plays	Alice			0.67	0.33					0.41	$= -\log \frac{2}{3}$
3	tennis	plays						0.5	0.5		0.69	$= -\log 0.5$
4	END	tennis								1	0	$= -\log 1$: a sure word
5	Alice	END	0.6	0.4							0.51	
6	plays	Alice			0.67	0.33					0.41	
7	golf	plays						0.5	0.5		0.69	
8	END	golf								1	0	
9	Alice	END	0.6	0.4							0.51	
10	sings	Alice			0.67	0.33					1.10	
11	END	sings								1	0	
12	Bob	END	0.6	0.4							0.92	
13	plays	Bob			1						0	
14	tennis	plays					0.5		0.5		0.69	
15	END	tennis								1	0	
16	Bob	END	0.6	0.4							0.92	
17	plays	Bob			1						0	
18	chess	plays					0.5		0.5		0.69	
19	END	chess								1	0	

Row 1: y_1 is Alice, and $\hat{Y}$ gives Alice 0.6 there.

$$\ell_i = -\log \hat{y}_{i,y_i}$$

Near 0 when the circled chance is near 1, large when it is near 0.

Training needs one number to make small. Take the average of the column ℓ: that is $\bar{\ell}$, the loss of the whole table.

	y	x	ℓ
1	Alice	END	0.51
2	plays	Alice	0.41
3	tennis	plays	0.69
4	END	tennis	0
5	Alice	END	0.51
6	plays	Alice	0.41
7	golf	plays	0.69
8	END	golf	0
9	Alice	END	0.51
10	sings	Alice	1.10
11	END	sings	0
12	Bob	END	0.92
13	plays	Bob	0
14	tennis	plays	0.69
15	END	tennis	0
16	Bob	END	0.92
17	plays	Bob	0
18	chess	plays	0.69
19	END	chess	0

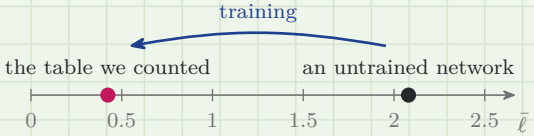
$\bar{\ell}$
0.42

$$\bar{\ell} = \frac{\ell_1 + \ell_2 + \cdots + \ell_n}{n} = \frac{8.05}{19} = 0.42$$

The table we counted scores $\bar{\ell} = 0.42$.

An untrained network gives every word about 1/8, so every ℓ_i, and $\bar{\ell}$ with them, is about $\log 8 = 2.08$.

Training makes $\bar{\ell}$ small: from about 2.08 down toward 0.42.



A first idea for one score: the network should make every circled chance big, so take minus their average. Call it $\tilde{\ell}$.

	y
1	Alice
2	plays
3	tennis
4	**END**

x
END
Alice
plays
tennis

Alice	Bob	plays	sings	chess	golf	tennis	**END**
0.6	0.4						
		0.67	0.33				
					0.5	0.5	
							1

$$\tilde{\ell} = \frac{1}{n}\left(-\hat{y}_{1,\text{Alice}} - \hat{y}_{2,\text{plays}} - \hat{y}_{3,\text{tennis}} - \cdots\right)$$

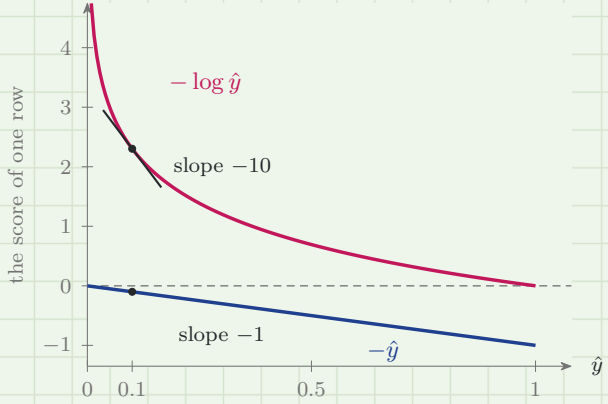
$$\bar{\ell} = \frac{1}{n}\left(-\log \hat{y}_{1,\text{Alice}} - \log \hat{y}_{2,\text{plays}} - \log \hat{y}_{3,\text{tennis}} - \cdots\right)$$

the score we use, with the logarithm

Both scores are smallest when every circled chance is 1: then $\tilde{\ell} = -1$ and $\bar{\ell} = 0$.

We take the logarithm. The two scores differ in how hard they push when a circled chance is small.

Training moves every weight along the slope of the loss: the steeper the slope, the bigger the push. Plot the score of one row against its circled chance $\hat{y}$.



Without the logarithm: the same slope, -1, wherever $\hat{y}$ is.

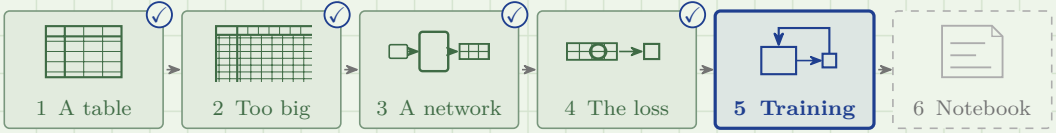
With it: steeper the further $\hat{y}$ is from 1.

At $\hat{y} = 0.1$ the slope is -10 with the logarithm and -1 without: ten times the push.

So a word that came next but got almost no chance gets the hardest push. Without the logarithm it gets the same gentle push as a word the network nearly had right.

Entry 5

Training



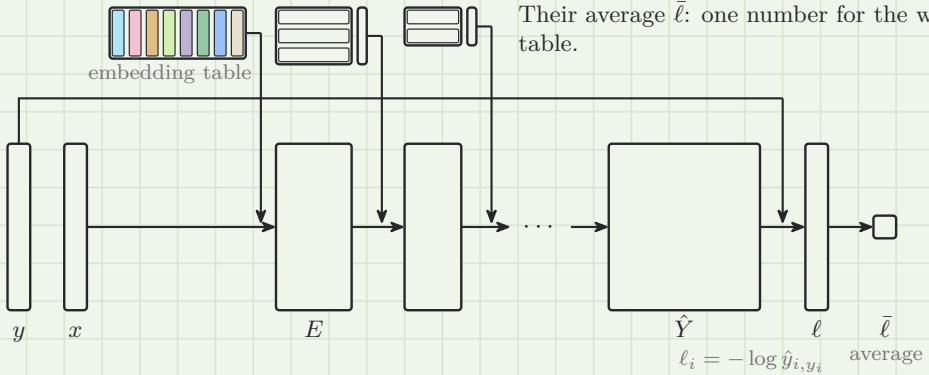
So far. The loss $\bar{\ell}$ scores $\hat{Y}$ with one number: the average, over the rows, of minus the log of the chance given to the word that came next.

Now. Training moves every weight of the network, the embeddings included, to make $\bar{\ell}$ small.

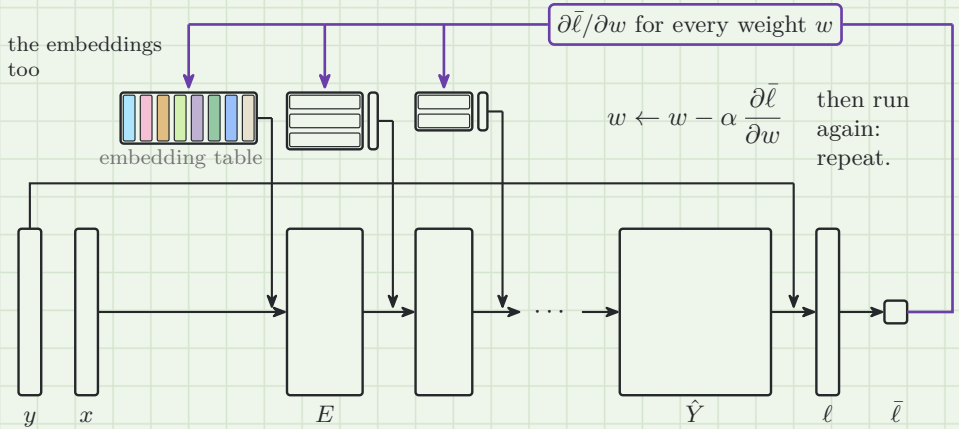
Parts 3 and 4 in one picture, as in the sketch. Left to right: the network computes $\hat{Y}$ from x, and y comes over the top to score it.

y comes over the top into the loss: row i of $\hat{Y}$ is scored against y_i.

Their average $\bar{\ell}$: one number for the whole table.

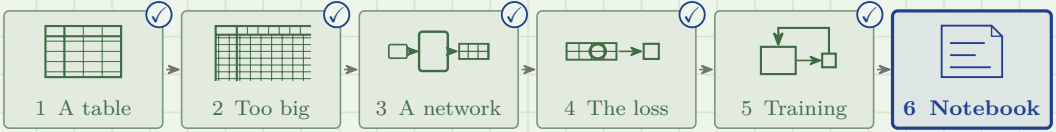


For every weight w, the gradient says how $\bar{\ell}$ changes when w moves. Each weight takes a small step α the other way, and the network runs again.



Entry 6

The notebook



So far. A network computes the table, the loss $\bar{\ell}$ scores it, and training moves every weight, the embeddings included, to make $\bar{\ell}$ small.

Now. A notebook builds such a network for a tiny language and trains it in seconds. You can run it yourself.

The notebook trains on a language of six words, *if*, *then*, *either*, *or*, *rain* and *sun*, with three rules after Noam Chomsky (1957).

rain and *sun* are sentences.
If A and B are sentences, so are *if* A *then* B and *either* A *or* B.

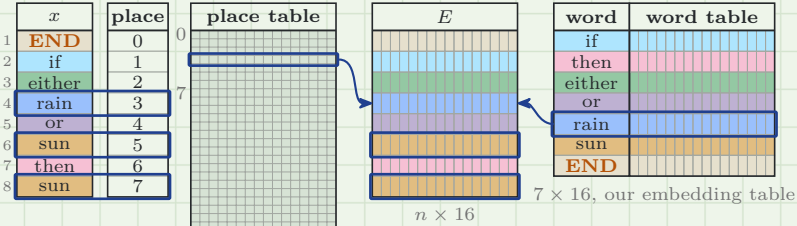
Sentences: *sun*, *if rain then sun*, *either sun or rain*, *if either rain or sun then sun*.

An *if* waits for its *then*, however many words come between. Chomsky used such sentences to argue that no machine with a finite memory can write English.

END is the seventh word of the dictionary. The notebook reads 3000 texts into one column with **END** between them, as we did. One of them, as y and x:

	y	x
1	if	**END**
2	either	if
3	rain	either
4	or	rain
5	sun	or
6	then	sun
7	sun	then
8	**END**	sun

The notebook keeps two tables of weights: a row of $d = 16$ numbers for each of its 7 words, and one for each place in a text, the distance of a row from its **END**.

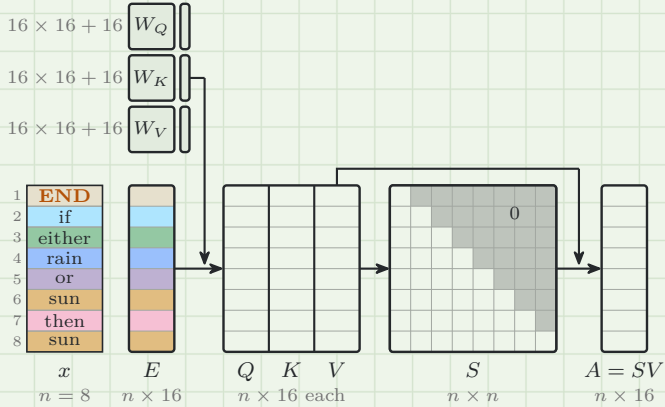


Row 4 holds rain, at place 3. Its row of E is the row of rain in the word table plus the row of place 3 in the place table.

Rows 6 and 8 both hold sun, at places 5 and 7, so their rows of E differ: a row knows how far it is from its **END**.

Both tables are weights, 112 and 640 numbers that start close to 0 and that training moves. At most 40 places: $n \leq 40$.

Attention lets a row read its prefix. Three blocks of weights turn E into queries Q, keys K and values V, and the queries and keys make the attention map S.



Each of W_Q, W_K and W_V comes with 16 biases: $Q = EW_Q + b_Q$, and K and V the same way.

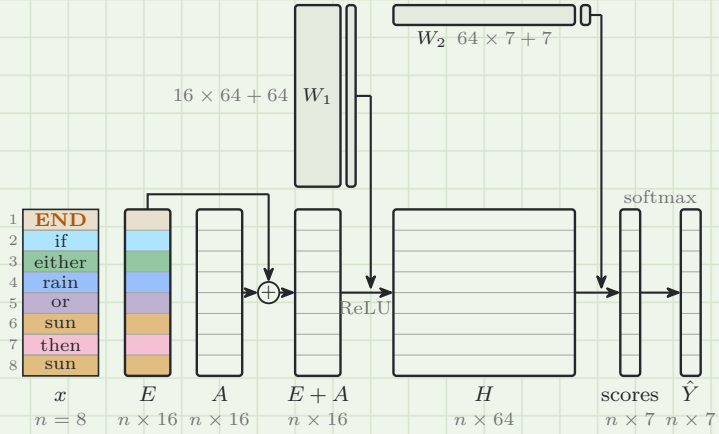
Row i of S compares the query of row i with the key of every row:

$$S = \mathrm{softmax}\big(QK^\top/\sqrt{16} + \mathrm{mask}\big)$$

The mask adds $-\infty$ above the diagonal: there S is 0, and row i reads only its prefix.

Row i of $A = SV$ averages the rows of V, weighted by row i of S.

Every row keeps its embedding and adds what it read. Two layers of weights then give it 7 scores, one per word, and the softmax turns them into chances.



First the sum, row by row: $E + A$.

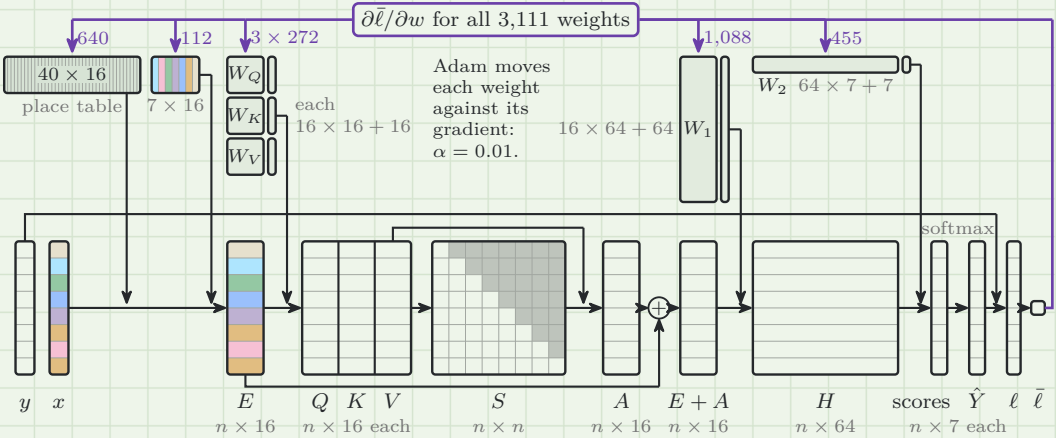
Then W_1, its 64 biases and a ReLU:

$$H = \mathrm{ReLU}\big((E + A)W_1 + b_1\big)$$

W_2 and its 7 biases give each row 7 scores, one per word: $HW_2 + b_2$.

The softmax turns each row of scores into 7 chances that add up to 1: the row of the table for its prefix.

A step runs the network on 32 texts, one at a time, and averages ℓ over all their rows into $\bar{\ell}$. Its gradient comes down into every block of weights, the tables too.



The rows that the untrained network computes for the same text. In every row the circle marks y_i, the word that came next.

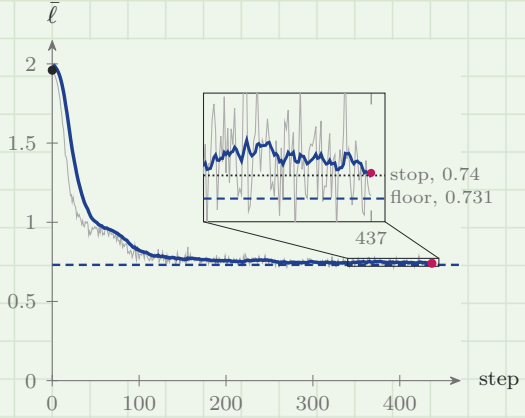
	y	x	if	then	either	or	rain	sun	END	ℓ
1	if	END	0.13	0.14	0.15	0.15	0.14	0.13	0.16	2.04
2	either	if	0.13	0.14	0.15	0.15	0.15	0.13	0.16	1.90
3	rain	either	0.13	0.14	0.15	0.15	0.14	0.13	0.16	1.97
4	or	rain	0.13	0.14	0.15	0.15	0.14	0.13	0.16	1.90
5	sun	or	0.13	0.14	0.15	0.15	0.14	0.13	0.16	2.04
6	then	sun	0.13	0.14	0.15	0.15	0.14	0.13	0.16	1.97
7	sun	then	0.13	0.14	0.15	0.15	0.14	0.13	0.16	2.04
8	END	sun	0.13	0.14	0.15	0.15	0.14	0.13	0.16	1.83

Every chance is close to $1/7 = 0.14$, and the rows are nearly the same: the tables start close to 0, so the network cannot yet tell one prefix from another.

So every ℓ_i is close to $\log 7 = 1.95$. For this text $\bar{\ell} = 1.96$.

On the whole corpus, $\bar{\ell} = 1.96$ as well. Training starts from here.

The notebook trains on its 3000 texts, x and y of 28,170 rows, on one thread of a laptop. A step picks 32 texts, computes $\bar{\ell}$ of their rows and moves every weight.



Before training every chance is about 1/7: on the whole corpus $\bar{\ell} = 1.96$, the dot at step 0.

The loss of each step, in grey, jumps around with its 32 texts.

Its running average, in pen, keeps 0.9 of itself and adds 0.1 of each new loss.

It falls below 0.74 after 437 steps, in under 10 seconds, and the loop stops. On the whole corpus $\bar{\ell} = 0.741$, the red dot.

A hair above the floor of 0.731, dashed: where a new sentence starts, up to four words are allowed, and chance decides among them.

The rows that the trained network computes for one text, *if either rain or sun then sun.* In every row the circle marks y_i, the word that came next.

	y	x	if	then	either	or	rain	sun	END	ℓ
1	if	END	0.44		0.45		0.05	0.06		0.82
2	either	if	0.26		0.23		0.24	0.27		1.47
3	rain	either	0.09		0.11		0.42	0.38		0.87
4	or	rain				1				0
5	sun	or	0.16		0.14		0.34	0.36		1.02
6	then	sun		1						0
7	sun	then	0.25		0.24		0.25	0.26		1.35
8	END	sun							1	0

Row 1, after **END**: if 0.44, either 0.45, rain 0.05, sun 0.06. The notebook opens its texts with these words 45, 45, 5 and 5 times in 100.

Where the rules force a word, the network gives it 1: *or* after *either rain*, *then* for the *if*, and **END** at the end.

Those rows cost 0. The rows with a choice keep a loss, as in our table: for this text $\bar{\ell} = 0.69$.

The attention map S of the same text after training. Row i holds the weights with which row i reads each row of its prefix, itself included.

	y	x
1	if	END
2	either	if
3	rain	either
4	or	rain
5	sun	or
6	then	sun
7	sun	then
8	END	sun

END	if	either	rain	or	sun	then	sun
1							
0.86	0.14						
0.87	0.13					0	
	0.04	0.91	0.04				
0.6	0.09	0.02	0.24	0.05			
0.02	0.82	0.04		0.12			
0.09	0.01	0.05	0.3	0.03	0.34	0.19	
	0.01	0.29	0.12	0.06	0.5		0.02
1	2	3	4	5	6	7	8

reads the row of

Above the diagonal every weight is 0: a row reads only its prefix. Each row adds up to 1.

Row 4 holds rain, and *or* is due. It reads row 3, its *either*: 0.91.

Row 6 holds sun, and *then* is due. It reads row 2, the *if* four rows up: 0.82.

Each row that must close a pair finds the word that opened it, however far back: what Chomsky's argument asks of a model. In another run these weights can come out different; the chances still follow the rules.

Like the table, the network writes by spinning its rows: the last row of $\hat{Y}$ for the text so far gives the next word, until **END** comes out.

Of 1000 texts it wrote, 995 follow the rules.

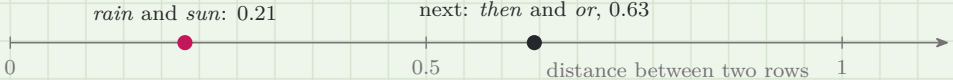
218 of them are not among the 3000 texts it learned from. A counted table writes only the texts of its corpus (part 1). Three of the new ones:

if either either rain or rain or sun then rain

either either rain or sun or if rain then sun

if either if rain then rain or rain then rain

Its embeddings learned too. In the trained word table, *rain* and *sun* end up nearest: nobody told the network that they play the same part.

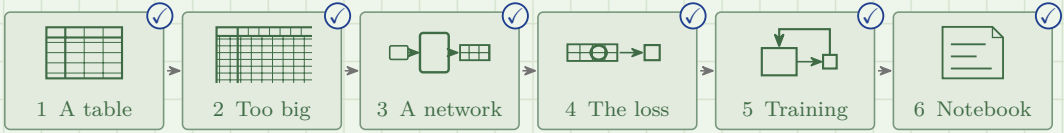


The notebook needs nothing installed and no GPU: it runs in a browser on Google Colab. On one thread of a laptop it takes under 20 seconds.

1. On the course site, `eth-aml-hs26.github.io/neural-networks`, find *A tiny language model*.
2. Press Open in Colab: the notebook opens in Colab, straight from the site.
3. Choose Runtime, Run all. The default CPU runtime is enough.

Inside, the names are the lecture's: `x`, `y`, `E`, `Y_hat`, `ell` and `ell_bar`, with $d = 16$ numbers per word. Read it cell by cell, and change it.

Six steps took us from a table of chances to a network that writes texts.



A tree of chances, then a table: one row per word of the corpus, one column per word of the dictionary, filled by counting.

A real table is far too big: a network computes each row when it is needed.

x becomes E through the embeddings, layers turn E into scores, the softmax turns them into $\hat{Y}$. Row i reads at most C rows of its prefix.

$\ell_i = -\log \hat{y}_{i,y_i}$, averaged into $\bar{\ell}$. The logarithm pushes hardest on bad guesses.

The gradient of $\bar{\ell}$ moves every weight, the embeddings too.

Chomsky's language: a network of 3,111 weights learns it in seconds and writes new texts that follow its rules.